Scaling Storage

Slides by Raed Almomani and Andrew DeOrio

Scaling road map

- Previously: Scaling static pages
  - DNS, CDNs
- Last Lecture: Scaling server-side dynamic pages
  - Round robin DNS, load balancing, hardware virtualization, containerization
- Today: Scaling storage
  - Database
  - Media uploads

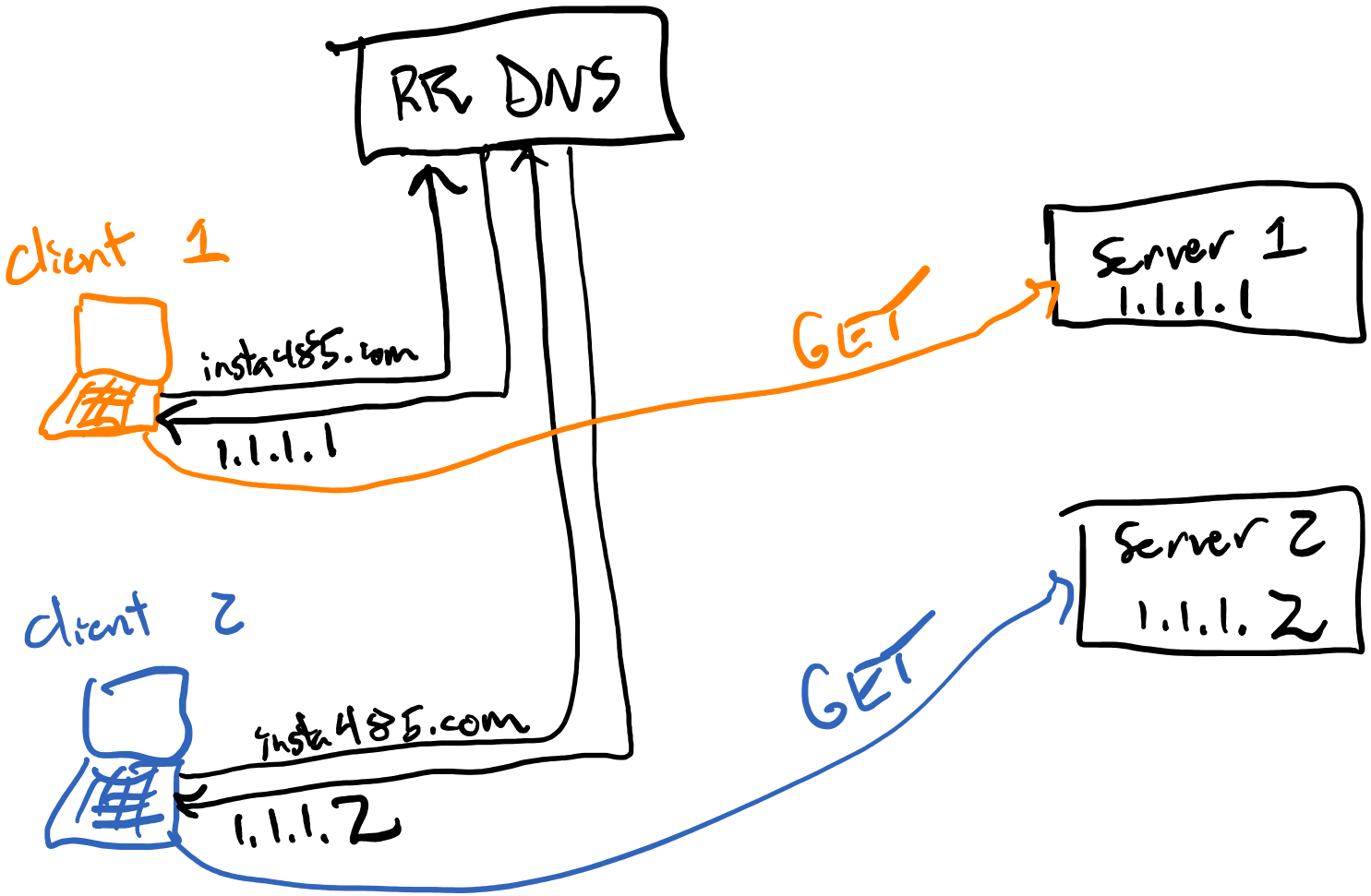
Agenda

- Distributed network database
- Sharding vs replication
- CAP Theorem
- Practical database examples
- Media storage

Review: Scaling dynamic pages

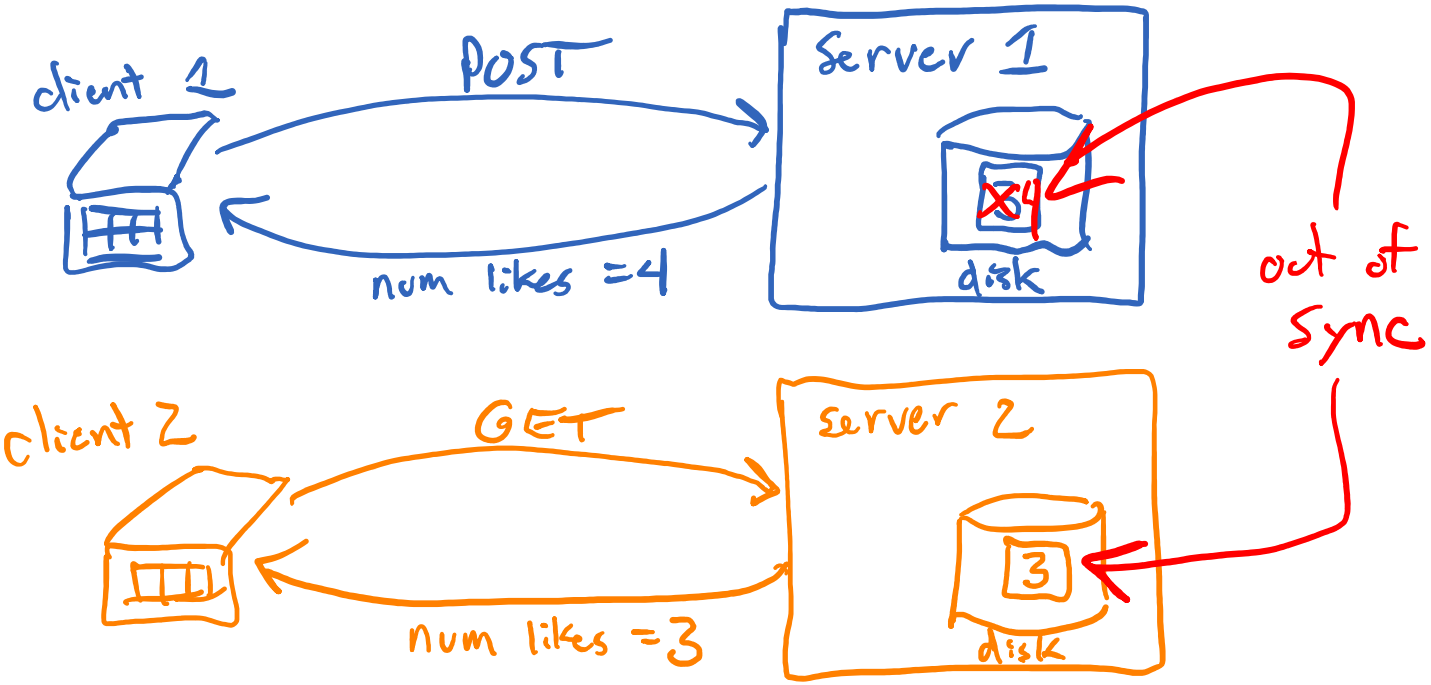
- Dynamic pages servers run in servers
  - AKA "Front end server" AKA "FE”
- Requests routed to servers via round robin DNS and load balancing

Review: Round Robin DNS diagram



Problem with many servers

- 2 clients connect to 2 servers, access the same data
  - E.g., one likes a post, and the other loads the same post



Many dynamic pages servers

- Different dynamic pages servers on every request
  - Physical machines, virtual machines, or containers
  - AKA "Front end servers”

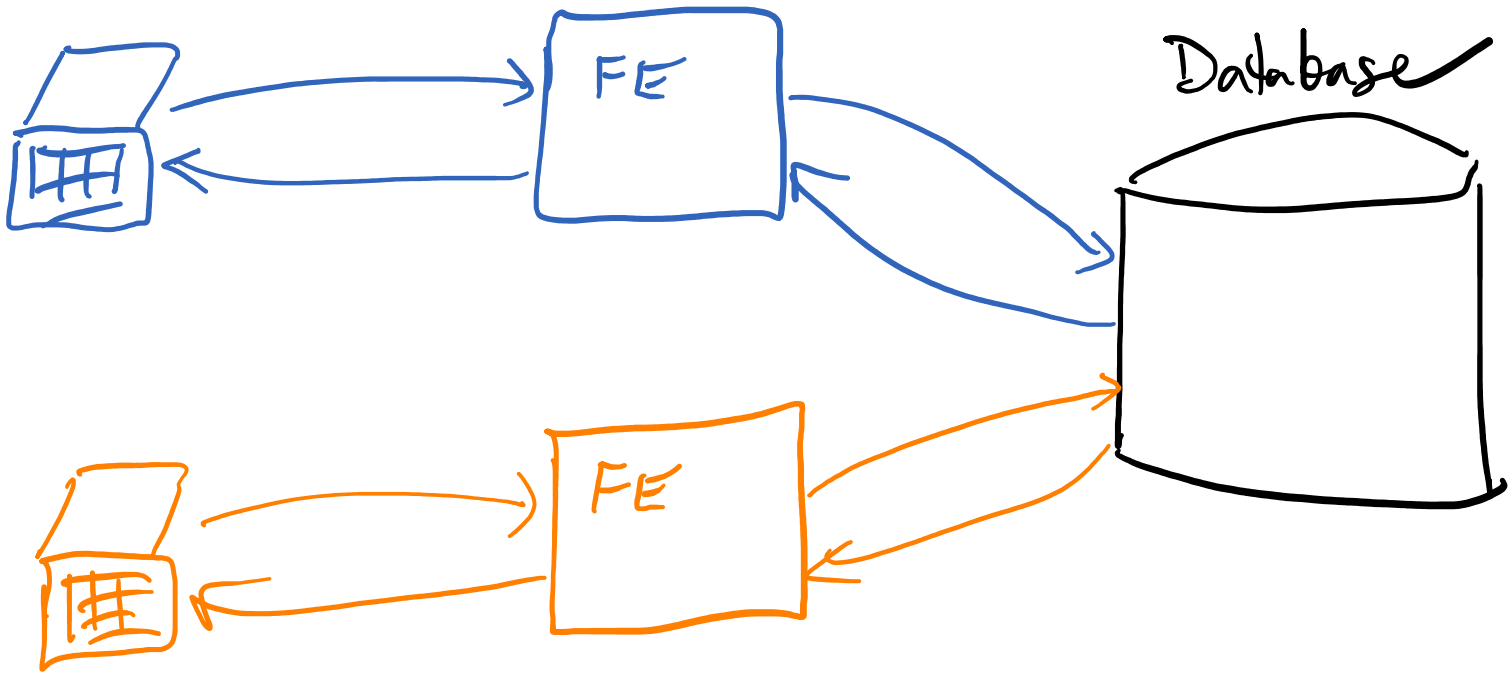
- **Problem**: How to synchronize dynamic pages servers?
- **Solution**: Dynamic pages servers are stateless, store everything in the database

Network database

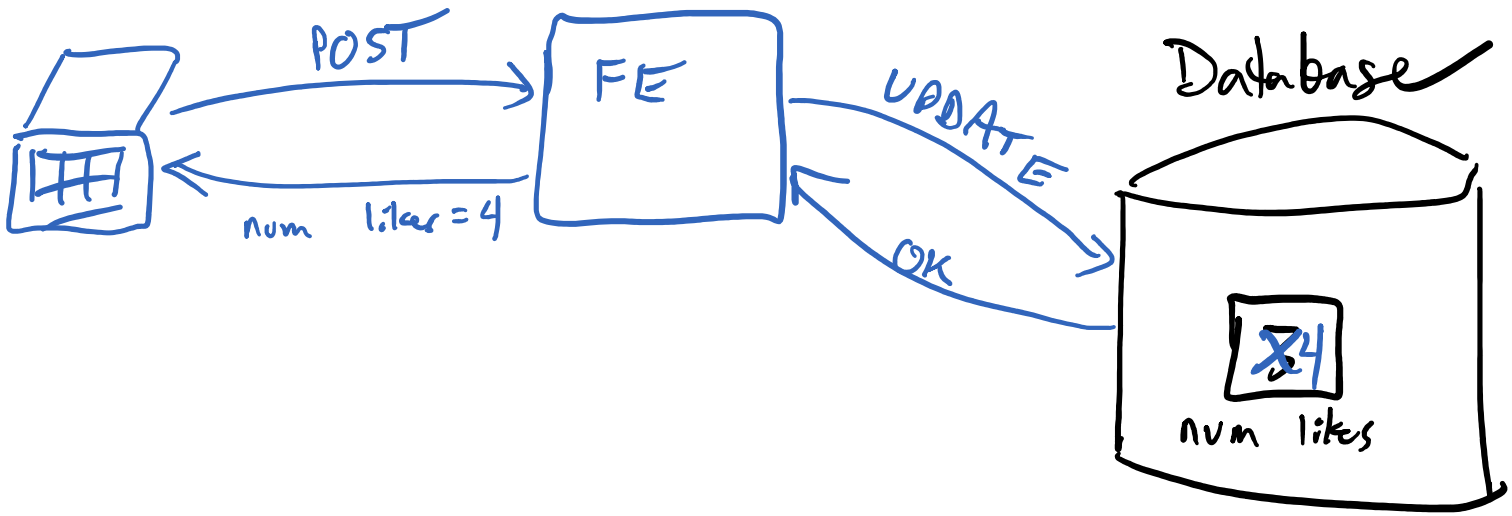
- Many dynamic pages servers make network requests to one network database
- *Network database:* Database software runs on a different server on the network
  - PostgreSQL, MongoDB, et al.
- *Centralized network database:* One server

Centralized network database

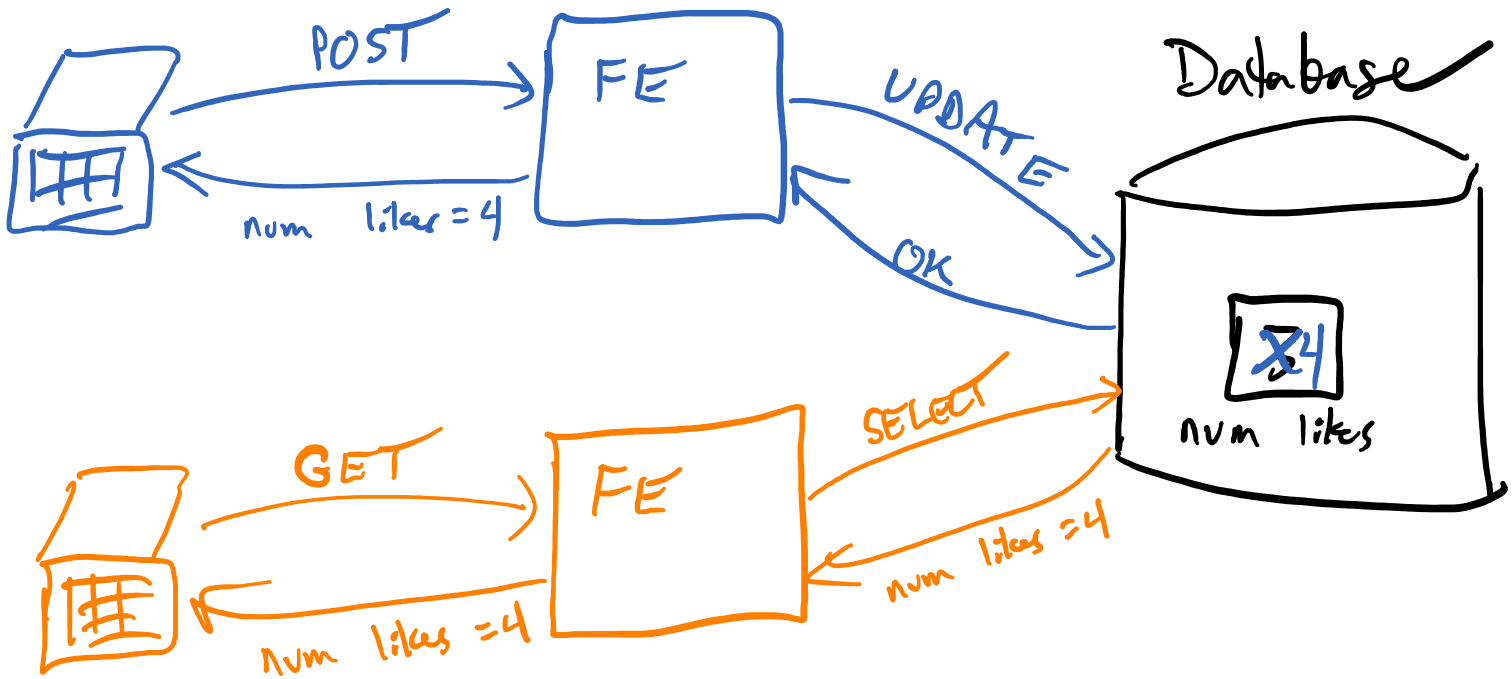
- *Centralized network database:* Single network database server



Centralized network database write



Centralized network database read



Centralized network database

- Data consistency is easy
- **Problem**: Performance bottleneck
- **Solution**: More database servers

Distributed network database

- *Distributed network database:* Many network database servers

- **Problem**: How to keep database servers in sync?
- **Solution**: Data consistency is hard, depends on which servers contain which data

- Problem: Which servers contain which data?
- Solution 1: Sharding by content
- Solution 2: Database replication

Agenda

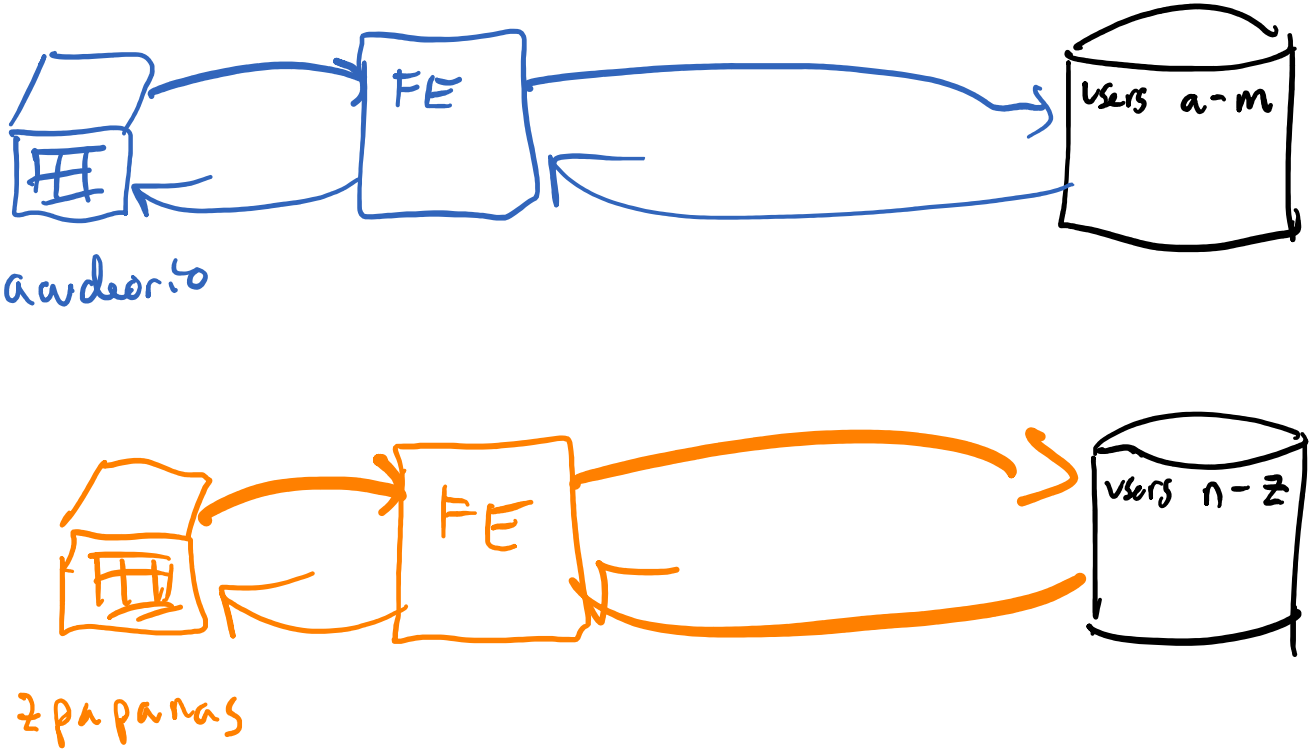
- Distributed network database
- **Sharding vs replication**
- CAP Theorem
- Practical database examples
- Distributed file system for media uploads

Sharding and replication

- Problem: Which servers contain which data?
- Solution 1: Sharding by content
- Solution 2: Database replication

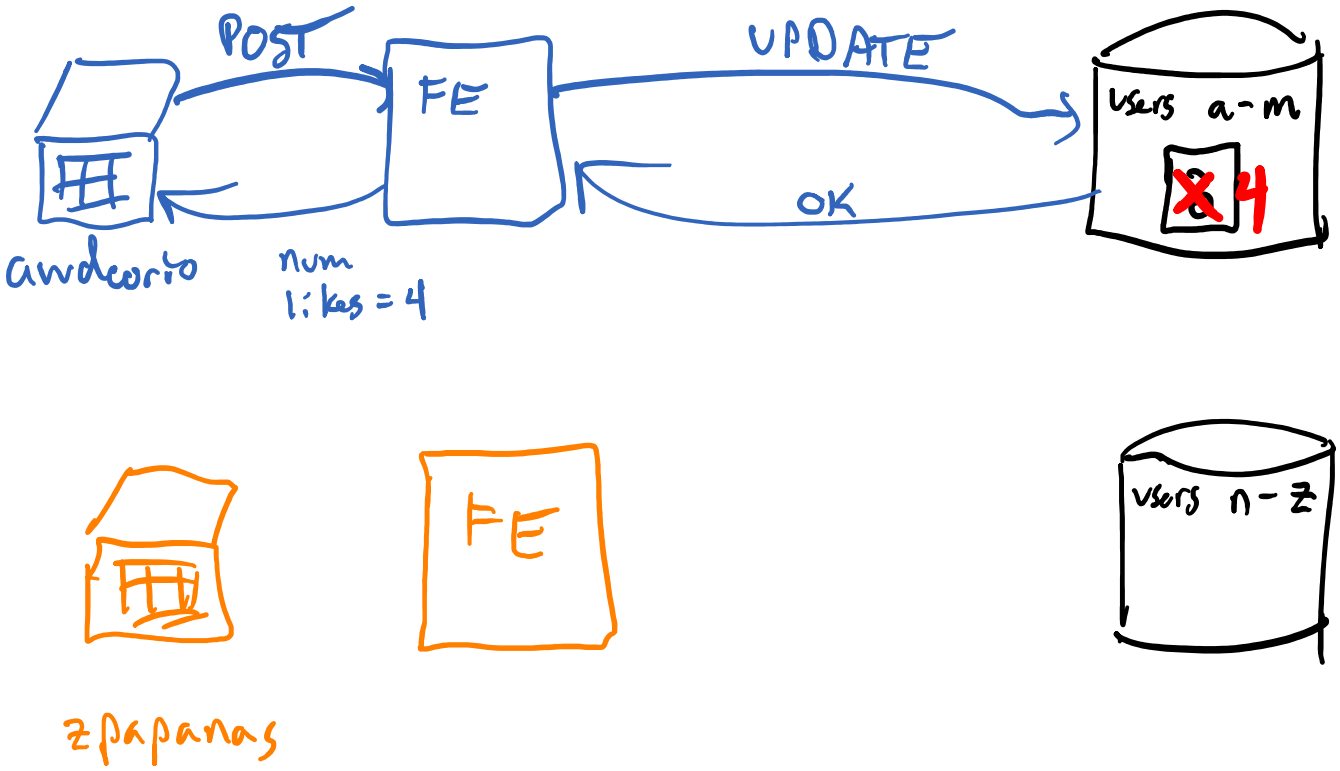
Sharding by content

- Different rows or tables on different DB servers



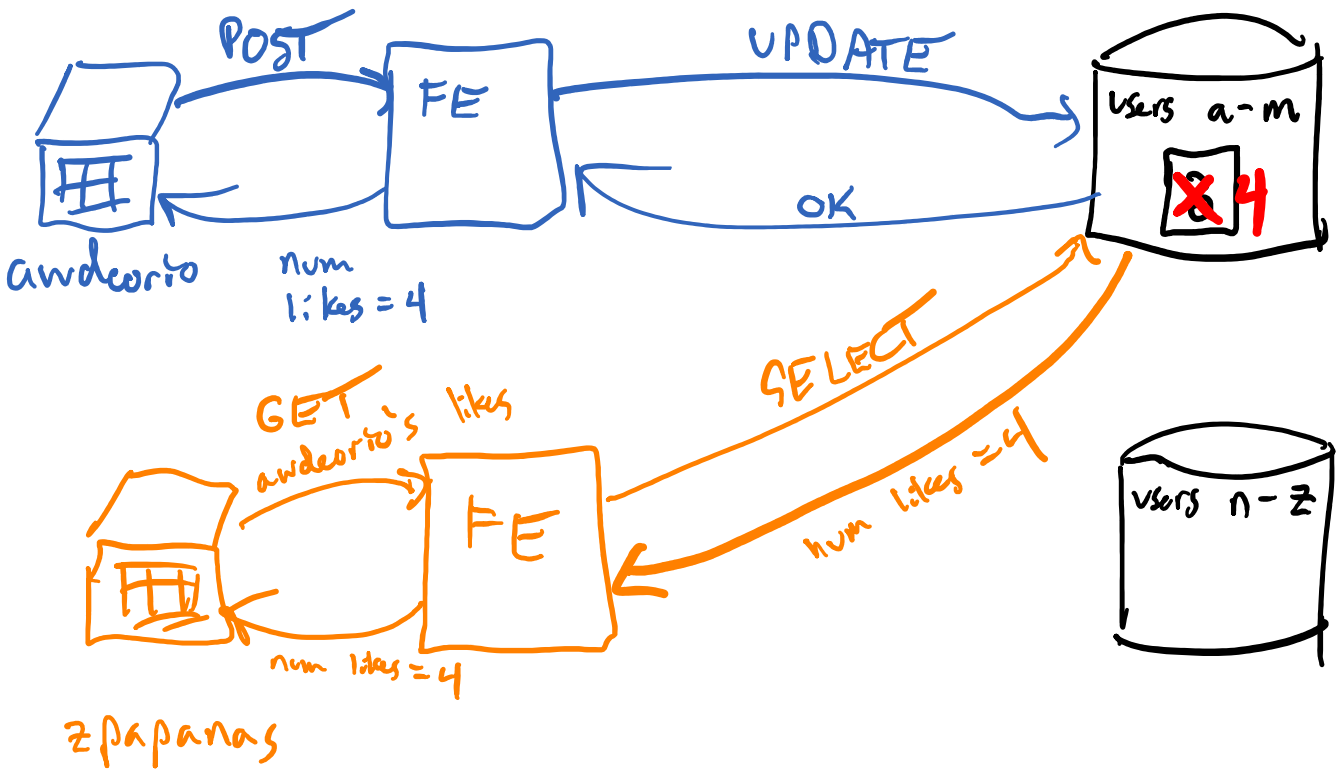
Sharding by content write

- A write modifies the *only copy*



Sharding by content read

- A read accesses the *only copy*

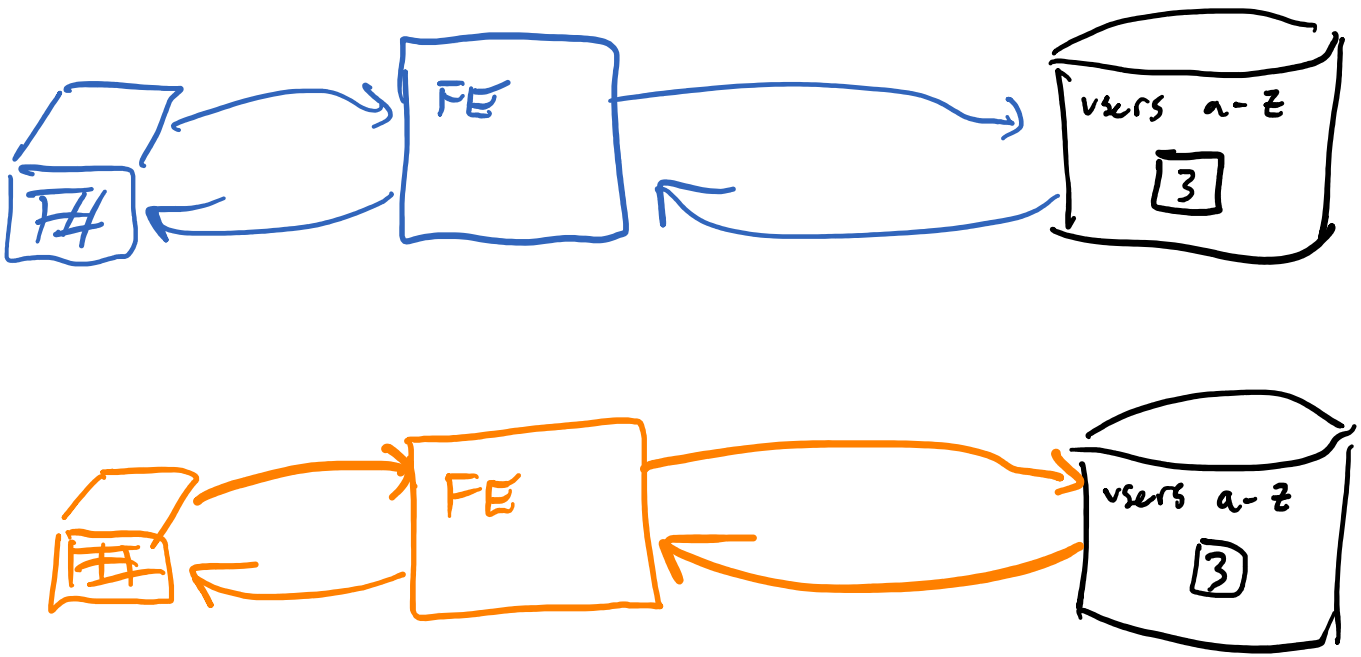


Sharding by content pros and cons

- Different rows or tables on different DB servers
- Strengths
  - Scalable because data is split among servers
  - Consistency is easier because no copies
- Weaknesses
  - Throughput: Single copy could be a bottleneck for frequently accessed data

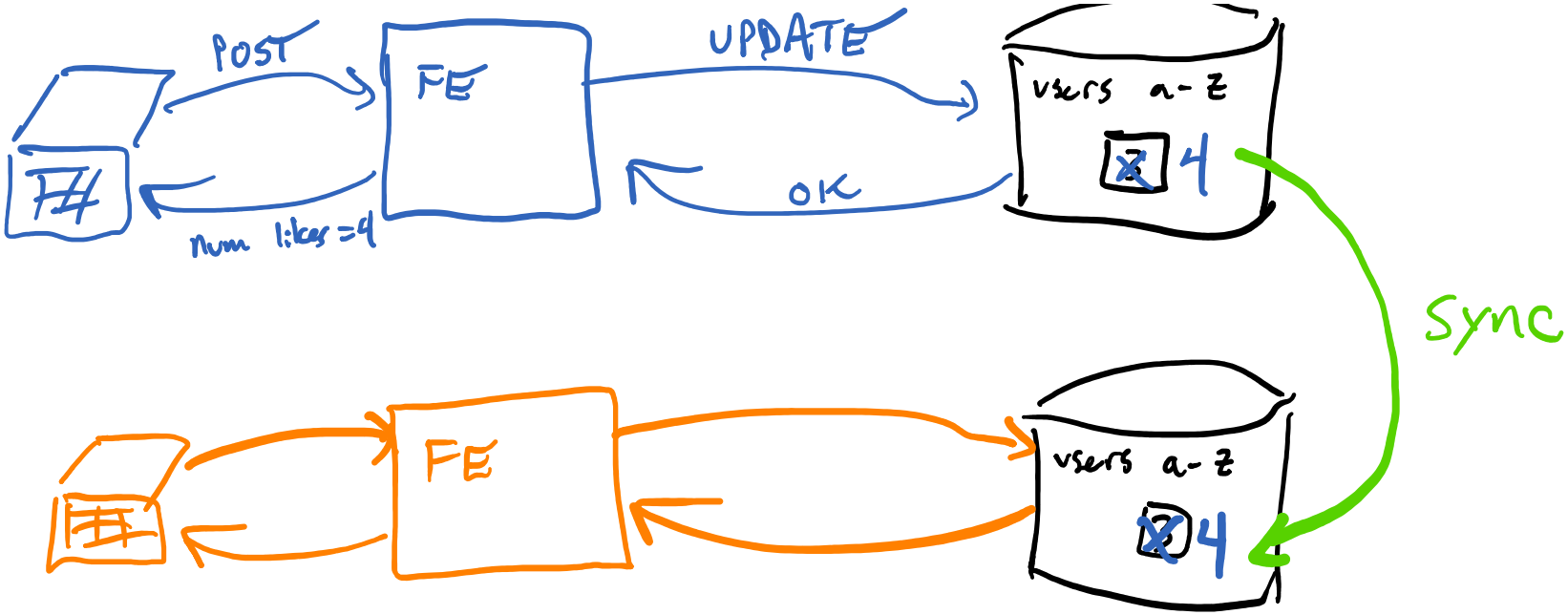
Database replication

- Multiple copies of the entire database



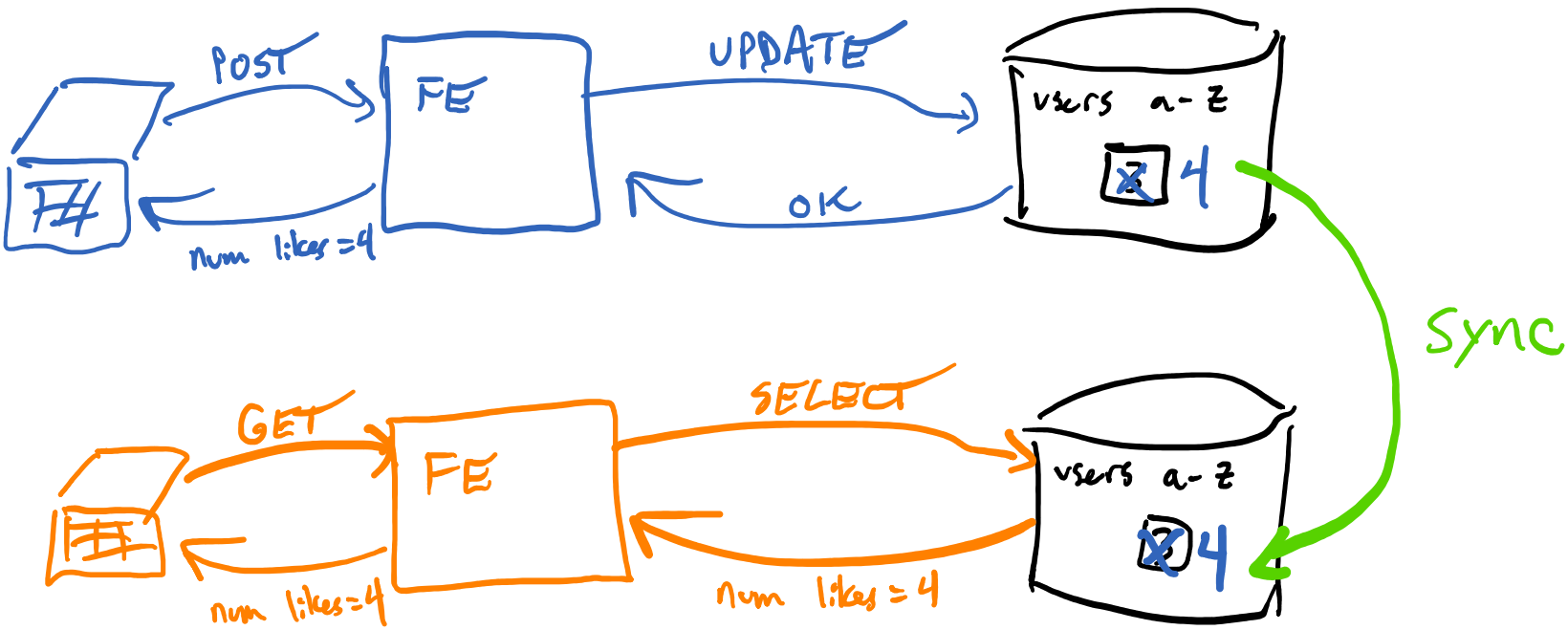
Database replication write

- A write can access any copy and must be synchronized to other copies



Database replication read

- A read can access any copy



Database replication pros and cons

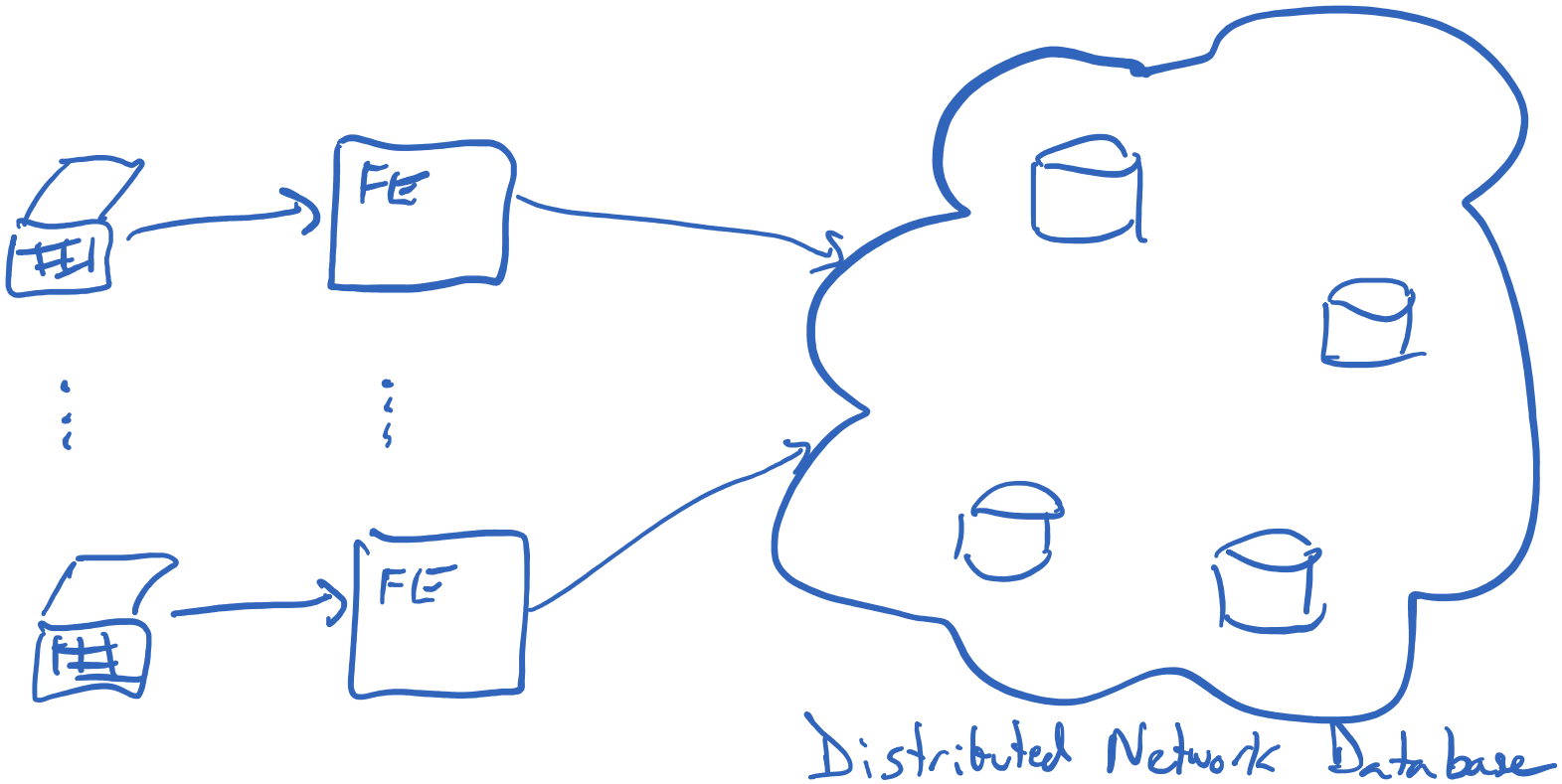
- Multiple copies of the entire database
- Strengths
  - Throughput: Avoid bottlenecks through multiple copies
- Weaknesses
  - Less scalable: Data might be too big to store on all servers
  - Consistency: Hard to keep copies in sync
    - Write to one needs to propagate to all the others
    - Moments of inconsistency, post shows up for some users and not others

Agenda

- Distributed network database
- Sharding by content, database replication
- **CAP Theorem**
- Practical database examples
- Distributed file system for media uploads

CAP theorem overview

- CAP theorem describes distributed database tradeoffs



CAP theorem: 3 properties

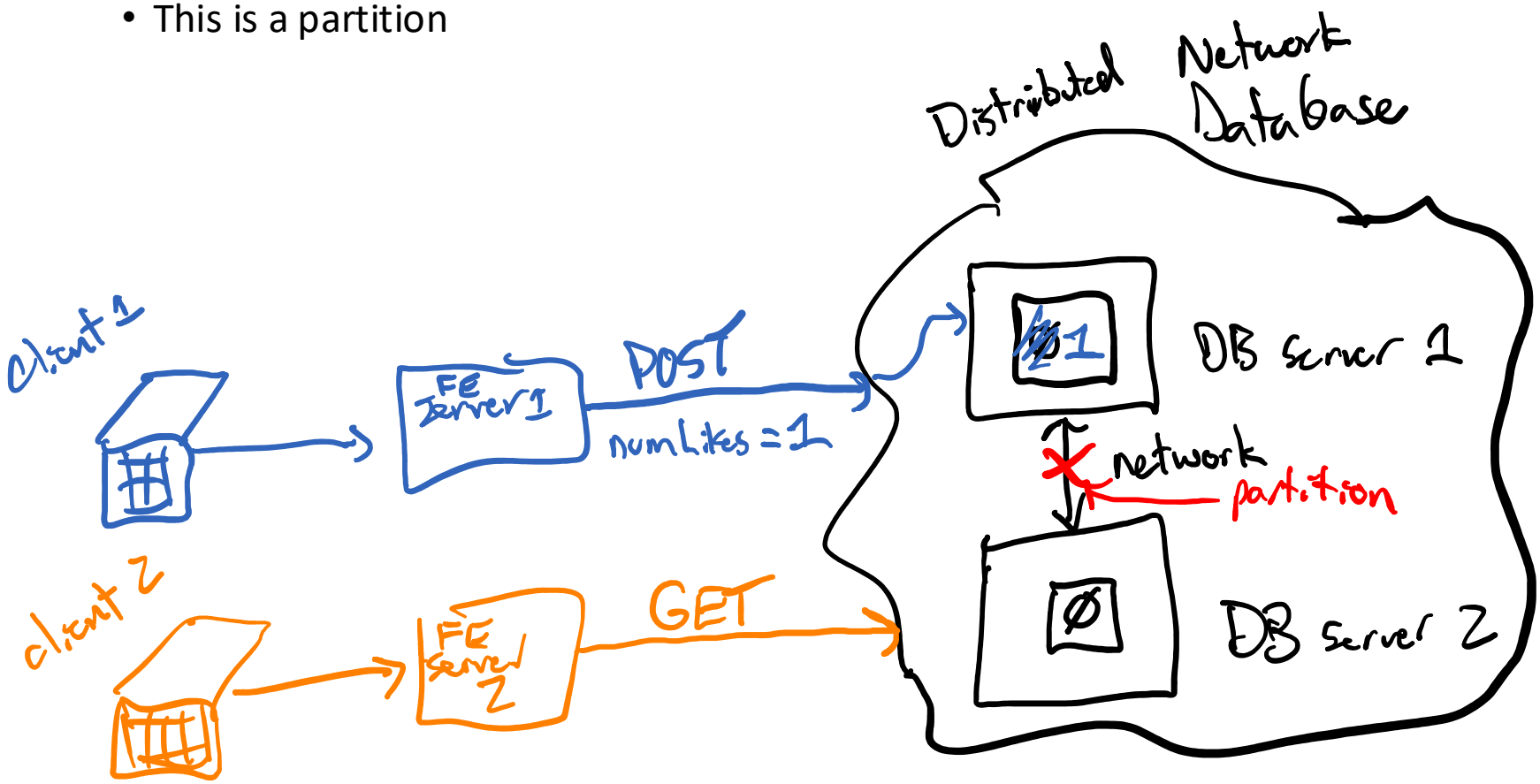
- ***Consistency***: Every read see latest write or an error
- ***Availability***: Every request gets a (non-error) response, without the guarantee that it contains the most recent write
- ***Partition-tolerance***: The system works despite network failure
- CAP Theorem: Pick 2 of 3

Network with no failures

- When the network is working correctly, you get Consistency and Availability
- Database servers can synchronize
- There is no partition

Network failure AKA partition

- When a network failure occurs, database servers cannot synchronize
  - This is a partition

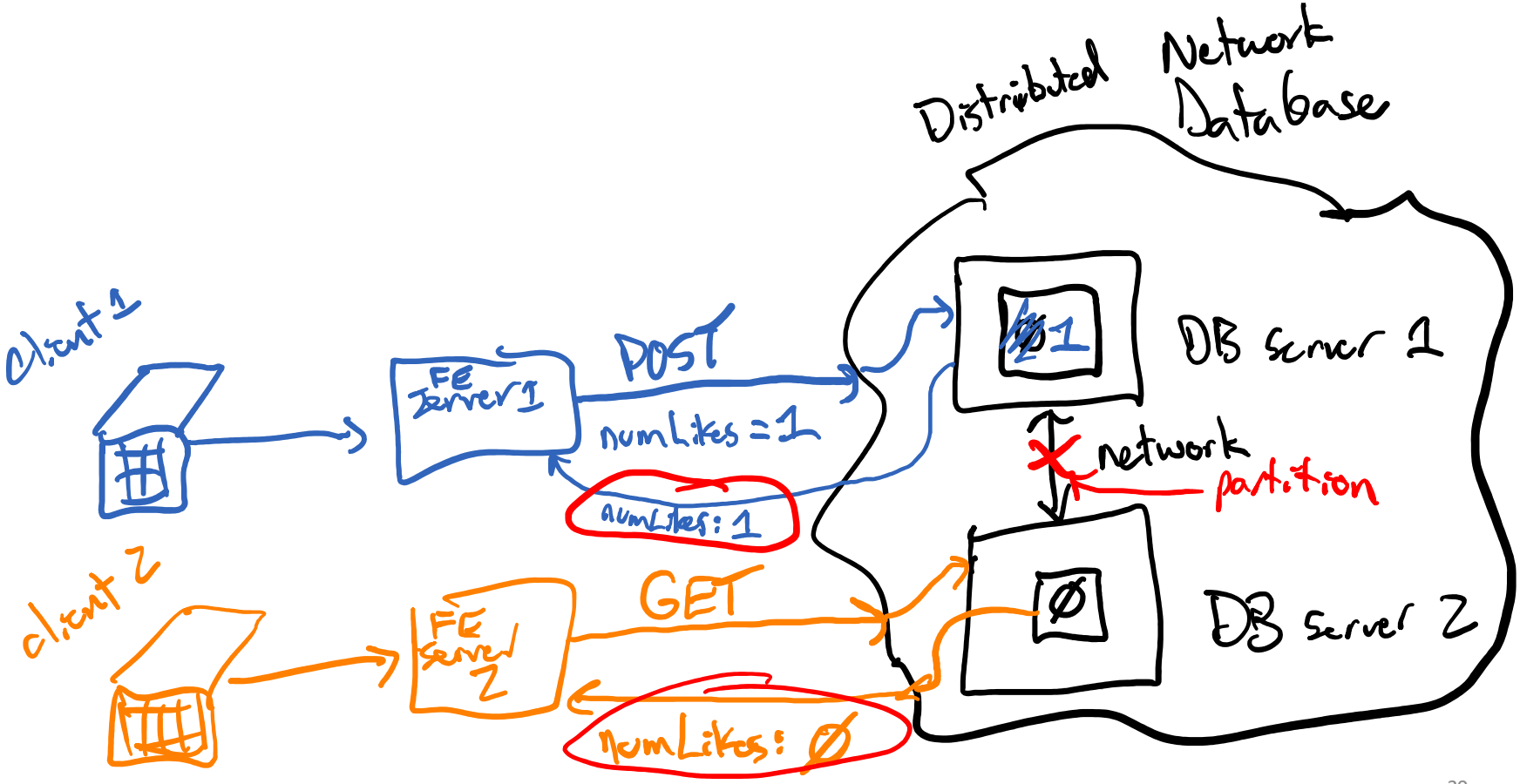


Network failure AKA partition

- When a network failure occurs, database servers cannot synchronize
  - This is a partition
- We have a choice
  - Return an incorrect value
  - Return an error

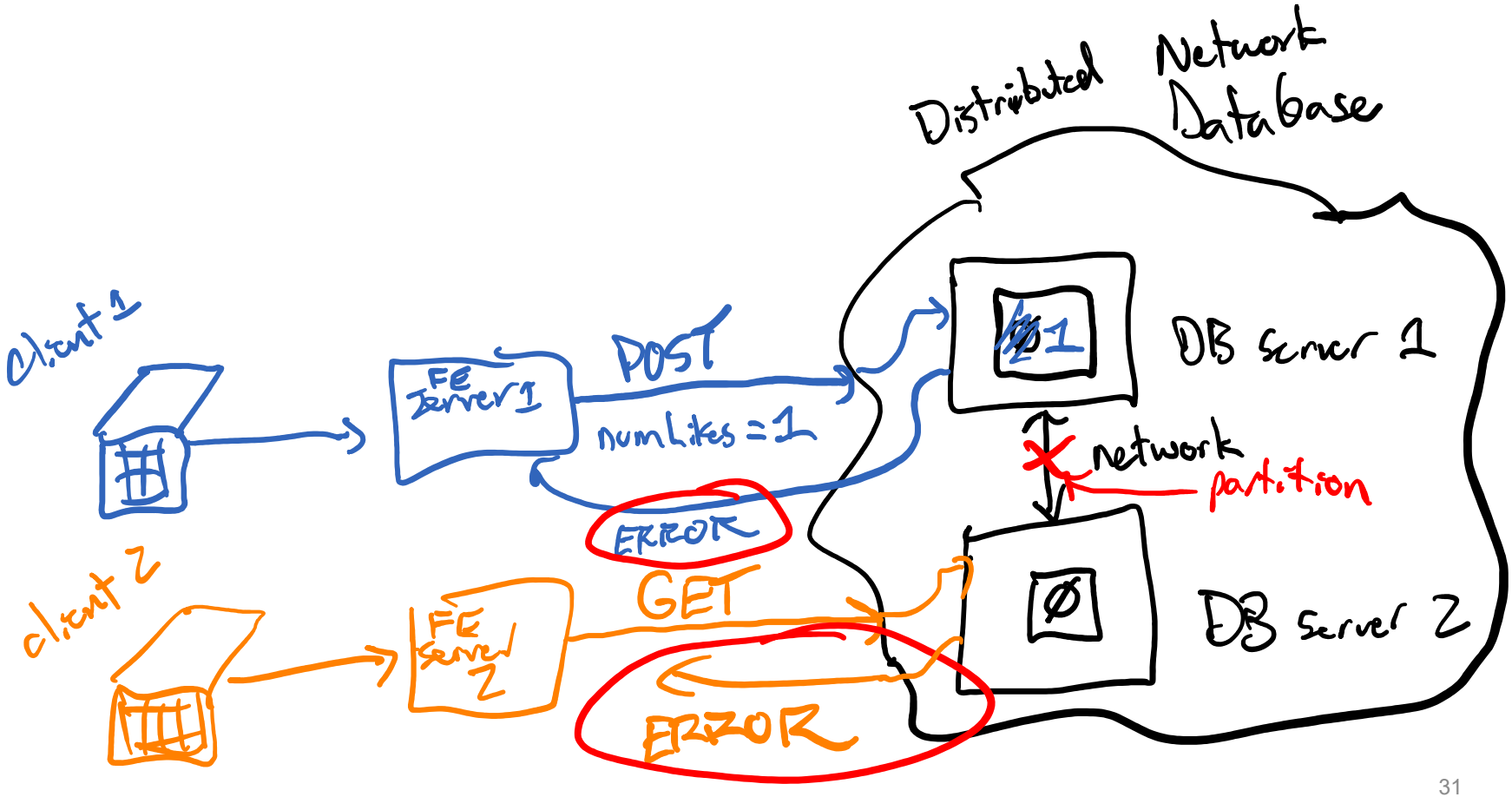
Available but not Consistent

- Return an incorrect value when network is partitioned



Consistent but not Available

- Return an error when network is partitioned



Which to choose?

- Prioritize consistency over availability
  - Relational databases, AKA RDBMS or SQL
  - PostgreSQL: Synchronous replication
  - Banking
- Prioritize availability over consistency
  - NoSQL databases
  - MongoDB: Primary is consistent, secondaries may be stale
  - Social media
- Avoid the problem
  - Buy a giant server for your DB so there's only one
  - Centralized network database

Agenda

- Distributed network database
- Sharding by content, database replication
- CAP Theorem
- **Practical database examples**
- Distributed file system for media uploads

PostgreSQL

- Traditional relational database
  - AKA "SQL"
  - AKA "RDMBS"
- Good for consistency
- Good for structured data

MongoDB

- NoSQL database
- Good for availability
- Good for unstructured data

Agenda

- Distributed network database
- Sharding by content, database replication
- CAP Theorem
- Practical database examples
- **Distributed file system for media uploads**

Media uploads and the database

- Media uploads: Instagram photos, TikTok videos, etc.
- Problem: Media uploads are expensive to store in a database
- Solution: Store to disk

- Write once, read many times. Don't need a database to maintain consistency.

Scaling media uploads

- Different dynamic pages servers on every request
  - Physical machines, virtual machines, containers
- Problem: How do dynamic pages server disks stay in sync?
- Solution: Dynamic pages servers are stateless. Store media uploads on network storage

Media storage implementation

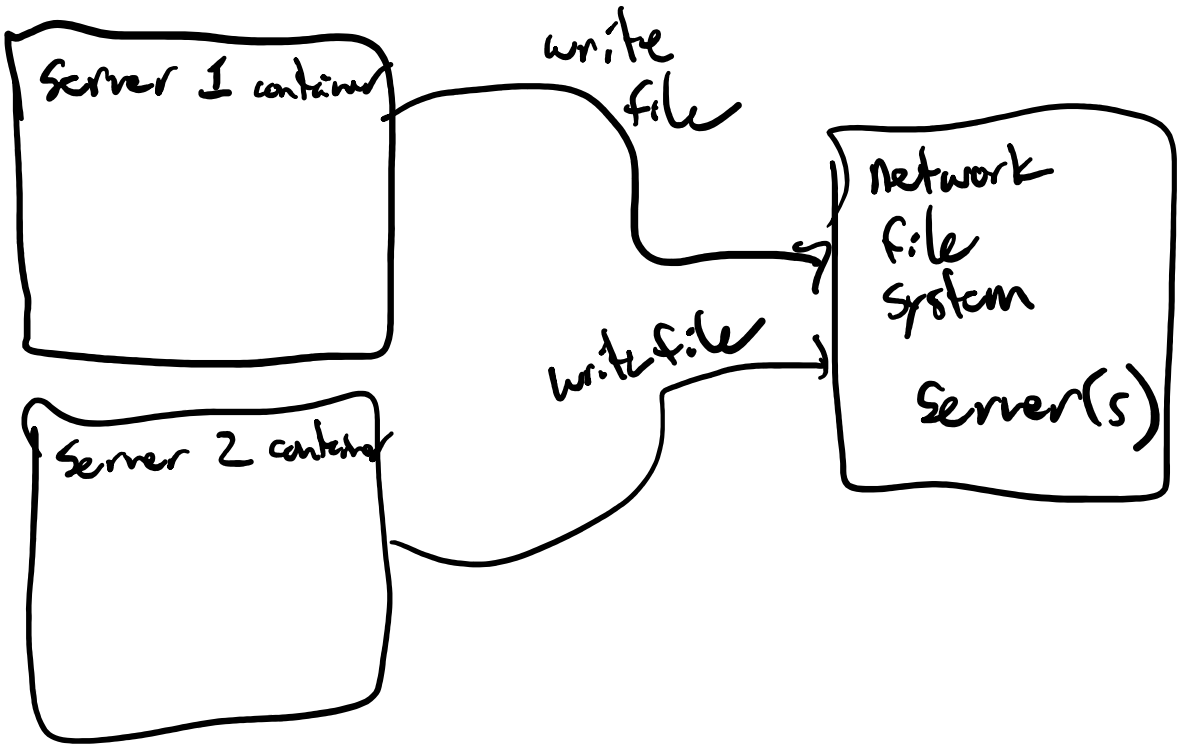
- Two subproblems for media storage
- Problem 1: Store the files
  - Many files
  - Large size
  - Exabytes of data (10^18)
- Problem 2: Serve the files
  - Many requests
  - Permissions control per-user

Store the files

- Problem 1: Store files
- Solution 1: Network file system
- Solution 2: Distributed file system

Network file system

- *Network file system:* Access files over the network much like local storage (hard drive)



Network file system PaaS

- PaaS managed network file systems

- AWS Elastic File System (EFS)
- Google Filestore
- Microsoft File Storage

Network file system pros and cons

- Network file system acts like a local file system
  - Near zero client code changes
- Problem 1: Speed and scalability
- Problem 2: Fault tolerance. What happens when network file system server goes down?

Distributed file system

- *Distributed file system:* Access files over the network, often with a special API
- Reliable, scalable file storage implemented with commodity hardware
- Google File System is an historical example

Distributed file system PaaS

- PaaS managed distributed file systems

- AWS S3
- Google Cloud Storage
- Microsoft Blob Storage

Distributed file system pros and cons

- Distributed file systems are more scalable and fault tolerant than network file systems
- Distributed file systems often use a special API
  - Requires client code changes

Network FS vs. distributed FS

- Technically, both are distributed systems
- Optimized for different things

- *Network file system* is optimized to be as similar as possible to traditional file system
- *Distributed file system* is optimized for scale and reliability on unreliable hardware

Connecting dynamic pages servers to distributed file system

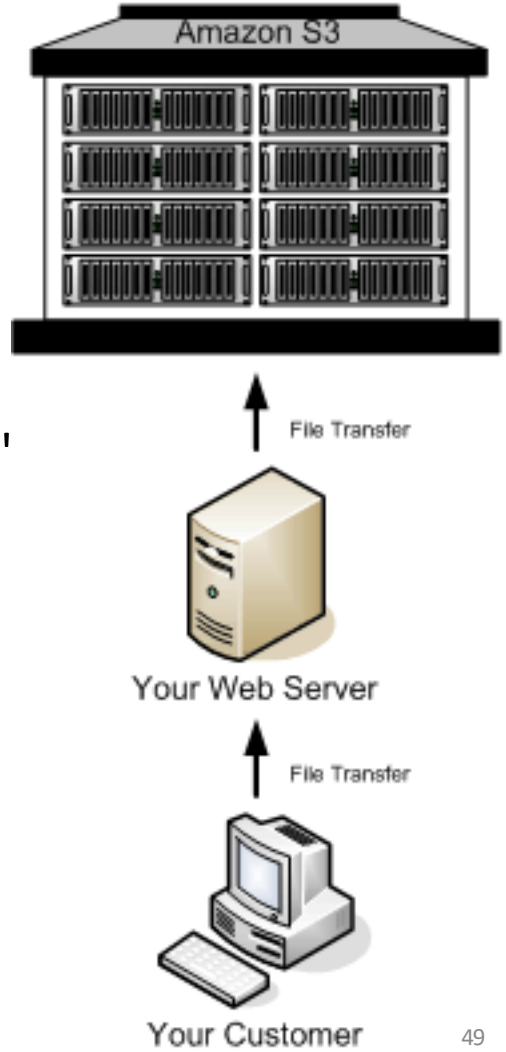
- Our data is now stored in a distributed file system
  - User connects to dynamic pages web server

- Problem: How do we upload and download?
- Solution 1: Dynamic pages server is a middleman between client and distributed file system
- Solution 2: Client connects directly to distributed file system

Media uploads proxy

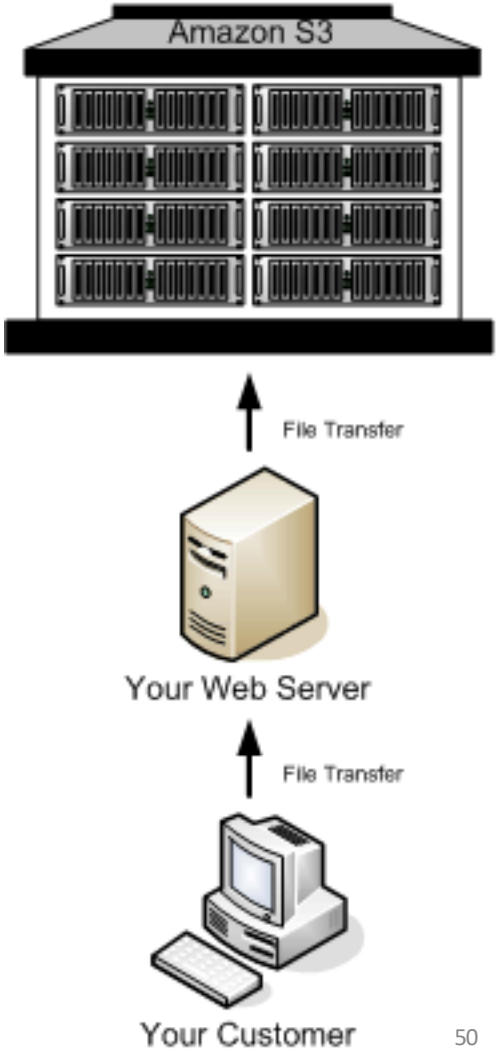
- Client sends upload to to dynamic pages server
- Dynamic pages server sends upload to distributed file system
  - Example: AWS S3

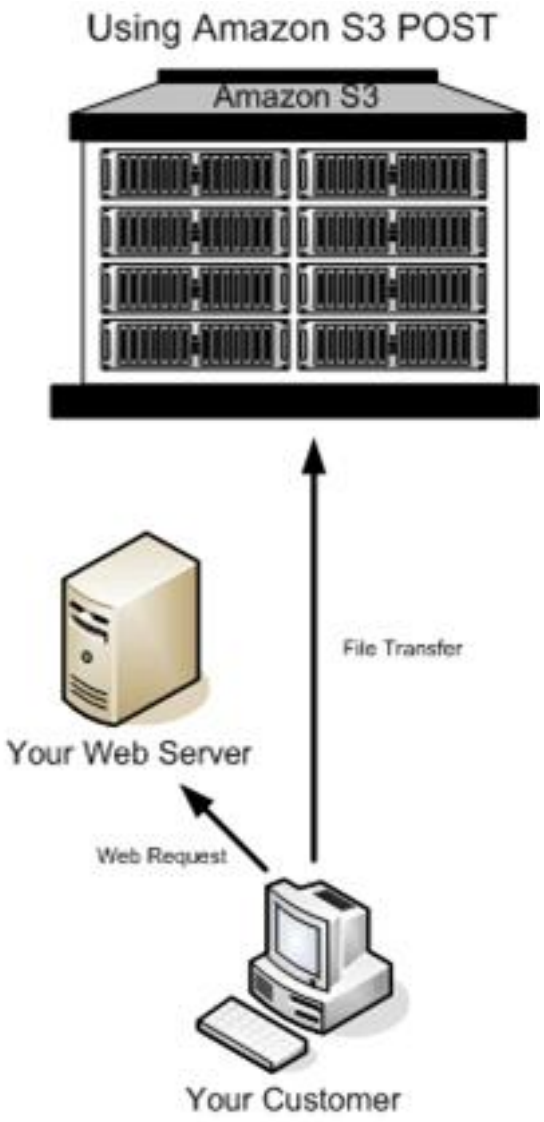
- Dynamic pages server is "Middleman" or "proxy"



Media uploads proxy pros and cons

- Easy to implement
- Problem: Large file transfer goes through multiple servers. Higher latency, higher bandwidth
- Solution: Direct upload



Direct upload

- Direct upload from client to distributed file system
- Dynamic pages server generates signed URL for client
- Client uploads large file directly to distributed file system